

PreTeXt for Instructors

PreTeXt for Instructors

Oscar Levin
University of Northern Colorado

August 6, 2026

Acknowledgements

This book started as a set of notes for a Professional Enhancement Program (PEP) at the 2023 Joint Mathematics Meeting ran by Steven Clontz and Oscar Levin designed to introduce mathematicians to PreTeXt. That guide was co-authored by the presenters. Since then, the notes evolved and were used in a number of similar workshops. While much of the content has been rewritten to focus on creating course materials in particular, large parts of the original text contributed by Steven Clontz remain.

The author would also like to thank the other members of the MathTech.org community, specifically Tien Chih and Chrissy Safranski for their feedback on the project as it develops.

About this guide

[PreTeXt](#) is a document authoring system that allows you to convert the source of your document into a variety of output formats, including fully accessible webpages, PDF, Epub, Jupyter Notebooks, and braille. This “write once, read anywhere” approach has made it a popular choice for authors of Open Educational Resources, but PreTeXt can also be used to create other kinds of mathematical documents as well. Recent updates make this process much easier; there has never been a better time to get started with PreTeXt.

The goal of this guide is to introduce instructors interested in using PreTeXt to create course documents, including lecture notes, slides, in-class activities, homework sets, and so on. We assume no familiarity with PreTeXt, but do assume you have previously used LaTeX to some extent for these tasks.

Specifically, readers of this guide will learn how to:

- Use GitHub Codespaces to create an editable PreTeXt document in their web browser.
- Write and structure content using PreTeXt markup.
- Add content to the document, including mathematics, graphics, interactive exercises, and more.
- Build accessible and interactive webpages as well as a static PDF from the same PreTeXt source.
- Easily deploy the interactive webpages online (for free).

We will also share tips for converting existing documents into PreTeXt.

Our Context. Think of this guide as a long tutorial for creating an entire set of course materials entirely in PreTeXt. Along the way, we will explore many of the features of PreTeXt. For example, we will walk through the process of creating a syllabus, and use that as an opportunity to consider how to divide a document into subsections, use various types of lists, include tables, and so on.

Of course readers should feel free to skip around to learn about particular topics as needed. Don’t forget that you can also use the official [PreTeXt documentation](#) as a reference for specific features as well.

Demo Course. If you would like to see what the final product of the tutorial looks like, visit the [Demo Course](#) on MathTech.org. The source code for the demo course is available on [GitHub](#). In the GitHub repository, you can view the source at various states of completion, corresponding to different stages of the tutorial, by selecting different branches.

Throughout this guide, we will indicate which branch corresponds to the completion of each section.

Contents

Acknowledgements	iv
About this guide	v
0 What is PreTeXt?	1
1 Setup	4
1.1 Creating a New Course.	4
1.2 Optional: Apply for your GitHub Education discount.	5
1.3 Optional: Installing PreTeXt locally	5
2 Workflow	6
2.1 Prerequisites	6
2.2 Build	6
2.3 Preview	7
2.4 Deploy.	8
2.5 Important: Saving your work	8
2.6 Collaborating	9
3 Organization	10
3.1 Course Structure	10
3.2 File Structure	11
3.3 First Edits	13
4 Course Components	15
4.1 Syllabus	15
4.2 Notes	18
4.3 Handouts.	18
4.4 Assignments.	18
4.5 Slides	18
5 Content	19
5.1 Images.	19

5.2	Video	19
5.3	Interactive Exercises	19
5.4	Tables	19
6	Publishing	20
6.1	The publication file	20
6.2	Showing and Hiding things	21
6.3	Numbering	21
6.4	Formatting	22
7	Managing Your Project	23
7.1	File and directory structure	23
7.2	Project.ptx	24
7.3	Versions	24
7.4	Building subsets	24
7.5	Codespaces, GitHub, and GitHub Pages	24
8	Converting To PreTeXt	25
8.1	Using Pandoc	25
8.2	Using the community	25
8.3	By hand	26
 Appendices		
A	Getting Help	27
B	Copyright and Licensing	29
C	Acknowledgement	30

Chapter 0

What is PreTeXt?

Before we jump in to creating a course in PreTeXt, we should try to understand the approach PreTeXt takes to creating documents, how it is different from other authoring systems, and why the choices PreTeXt makes are beneficial for authors of course materials.

First, PreTeXt is a **markup language**: you write content in plain text files, using special strings to indicate structure and meaning. These plain text files are then transformed into various output formats that look nice and are easy to read. In this way, PreTeXt is similar to L^AT_EX, as well as other markup languages such as HTML and Markdown. And while there is work being done to create a WYSIWYG (What You See Is What You Get) editor for PreTeXt, the primary way to create documents in PreTeXt is to write the source files directly.

When you first start using a markup language, all the extra typing to indicate structure can feel overwhelming. To emphasize text, you have to type extra characters (in PreTeXt, you would wrap the emphasized text in `<em>` tags; or in L^AT_EX, you would use `\emph{}`). This may seem more difficult than hitting `CTRL`+`I` in a word processor to italicize text, or clicking on a button in the interface, but once you get used to it, and use autocomplete, many people find it much easier. Additionally, features of modern text editors, like VS Code, can make it much easier to edit documents quickly (for example, in VS Code you can use `ALT`+`UP` to move an entire line of text up without needing to select, copy, and paste it).

Semantic vs Visual Markup. Another advantage of using a markup language is that it *should* help you focus on writing your content without getting distracted by making sure it looks a particular way. When this is achieved, it can speed up writing and make us better communicators. We admit that actually letting go of formatting focus can be difficult: we have all learned how meaning is communicated visually, and it is tempting to try to mimic similar formatting as a proxy for communicating our meaning clearly. A few examples might be in order.

Consider the case of formatting references. Different publishers have made different decisions about their preferred way to format references, both inline and at the end of a document. It is uncontroversial that removing the burden of remembering how to format every reference correction is a huge advantage. Similarly, when cross referencing a theorem, are we, the authors, the best judges of whether “Theorem 7.2” should be capitalized or bold or underlined? These are decisions that *publishers* should make once and let us simply use their style. Authors should just indicate that the text is a reference. Another

example: when defining a term, it is customary to indicate the words being defined using some special formatting. So how should an author indicate this? We don't want to be required to remember that such words should be marked as bold or italic, but rather we should proclaim that the words are **terms** (in PreTeXt, this is done using a `<term>` tag).

Both PreTeXt and L^AT_EX allow for this **semantic markup**, where authors indicate the meaning of the text rather than its appearance. Unfortunately, L^AT_EX is so powerful that it is very easy to extend it to include purely visual formatting markup. This leads to documents that are impossible to convert to accessible formats, since the *intended* meaning of the visual markup is obscured. If a particular author has decided to mark both terms and emphasized text using `\textit{}`, how can we infer meaning to produce a useful braille version, for example? Or perhaps a more common example: if an author specifies a page break, what should that look like when the document is viewed as a web page?

With very few exceptions, PreTeXt forces authors to use semantic markup. This makes it much easier to produce multiple output formats from the same source files, and ensures that documents are accessible to all readers. This does require a shift in thinking about how we write. If we ever find ourselves asking “How do I make this text look like ...?”, we should pause and instead ask “Why did I want to make the text look like...?” Once we have answered that question, we can use the appropriate semantic markup to indicate the intended meaning, and let the output format decide how best to represent it visually (or non-visually, to a visually impaired reader).

Source Format. PreTeXt source files are written in XML (eXtensible Markup Language), a widely used markup language for storing structured data. XML is a very flexible language, and PreTeXt defines a particular **schema** (a set of rules) describing which document parts can or must contain which other parts.

Think of a document as a collection of nested boxes. A book contains chapter boxes, which may contain section boxes. A chapter or section contains text, but that text is organized into paragraph boxes, theorem boxes, and so on. It makes sense for a section box to hold a lemma box, but you would not expect a section to be inside a lemma. You also can't have a section start in one chapter and end in the next. XML allows us to represent this nested structure clearly, and the PreTeXt schema defines which boxes can go inside which other boxes. Being required to follow good document structure is another way that PreTeXt helps authors focus on creating well organized and understandable content.

Practically, PreTeXt source files look something like the following.

Listing 0.0.1 Sample PreTeXt Markup

```

<section xml:id="sec-great-title">
  <title>Great Title</title>
  <p>
    This is a simple paragraph before a theorem.
  </p>
  <theorem>
    <statement>
      <p>
        <pretext/> is awesome.
      </p>
    </statement>
    <proof>
      <p>
        Obvious.
      </p>
    </proof>
  </theorem>
</section>

```

The “sides” of the boxes are represented by **tags**, each a word inside angle brackets. An opening tag, like `<theorem>`, indicates the start of a box, and a closing tag, like `</theorem>`, indicates the end of that box. Some tags are self-closing, like `<pretext/>` which indicates an empty box; the box *is* the content, not a container for contents (in this case, the tag is a macro that expands to “PreTeXt”).

Some tags can have **attributes**, like the `@xml:id` attribute in the opening `<section>` tag above (xml-ids provide a code to be used by a cross reference, much like the `\label{}` markup in $\text{\LaTeX}$). It is conventional to refer to attributes in our documentation using an “@” sign before the name of the attribute. However, in the actual source that we write, we don’t type the “@” symbol.

When you start out writing PreTeXt, all this extra markup seems like a lot. We promise that with practice and by using a good text editor with autocomplete features, the extra typing is not as bad as it seems. You will soon find your eyes skipping over the markup parts and focusing on the content itself. Remember the learning curve you experienced with $\text{\LaTeX}$. It will be okay, and totally worth it.

Building. Like $\text{\LaTeX}$, we will need to transform our PreTeXt source files into output formats that are easy to read. We call this processing of source into output **building** the document. In [Chapter 2](#) we will cover the steps to build, view, and **deploy** projects.

This brings up another difference with $\text{\LaTeX}$: default setup for PreTeXt documents is to think of each document as a **project**, consisting of a folder containing all the source files, external assets (like images), and configuration options needed to build the document. It will also contain the output files, in their own subfolders of **output** (web output will contain many HTML files, images, and other assets; PDF output will be a single `.pdf` file). This project-based approach makes it easy to manage all the files needed for a document in one place, and makes it easy to share the entire document with others (for example, by hosting the project folder in a GitHub repository).

Chapter 1

Setup

Authoring in PreTeXt requires nothing more than a wifi connection and GitHub account. GitHub users also can share their content for free on GitHub Pages!

If you prefer to use a local setup instead of the cloud-based interface, this is absolutely possible, but requires a bit more setup. A list of the steps you will need to complete are given in [Section 1.3](#), but remember, this is entirely optional.

If you do not already have a GitHub account, follow the instructions on [GitHub's signup page](#).

1.1 Creating a New Course

We are going to create a **repository** on GitHub that will hold all our course documents.

Go to <https://github.com/pretextbook/pretext-codespace>. On that page, below a short list of files, you will see a README that gives instruction for creating a project from that template.

Once you have a repository created, you will start up a codespace by clicking the green Code button near the top right of the repository page, and then selecting the Codespaces tab (not Local) and then Create codespace on main. This will create a new codespace for you, which may take a few minutes to start up the first time.

The advantage to starting from this template is that it contains a `devcontainer.json` file that tells the GitHub Codespace to install the software PreTeXt needs to run. In particular, once the Codespace is up and running, you can initialize a new PreTeXt course as follows.

1. Open the **command pallet** in VS Code by pressing Ctrl+Shift+P (or Cmd+Shift+P on a Mac). Start typing “PreTeXt: New Project” and select that option when it appears.
2. Select “Course” from the available options.
3. You will be prompted to provide a location for the project. The default path, which should be `/workspaces/[repository-name]`, is what you want to select.

The page will now refresh and when it does, you will see some new folders and files in the file explorer on the left side of the screen. These are the files that make up your new PreTeXt course!

You can now proceed to [Chapter 2](#). The remainder of this chapter contains optional steps for upgrading your GitHub account and installing PreTeXt locally if you prefer to not use the cloud-based interface.

Demo Course Code

Completing the steps above should put your folder in the same state as the `1-initial-setup` branch of the [Demo Course repository](#).

1.2 Optional: Apply for your GitHub Education discount

While you can use GitHub for free, there are some useful features that require a “pro” account. The main advantage in this context is that the free Pro account you get access to will allow you to use GitHub Pages to publish your PreTeXt content while still keeping your repository and source files private. This can be useful if you want to use a single repository to hold both your assessments and student facing documents.

Educators and non-profit researchers can get a coupon for a free upgrade to the pro level. Apply at [Education.GitHub.com](https://education.github.com) to unlock these features (in our experience, applications are usually processed quickly for .edu email addresses).

1.3 Optional: Installing PreTeXt locally

If you would rather author PreTeXt locally on your own computer (which does offer advantages of not requiring an active internet connection, and the security of having your files stored on your device), we suggest one of the two following approaches.

1. Run all the software PreTeXt needs inside a **docker container**. You will have the exact same set of software used for the browser-based authoring experience. This is easier to set up, but will likely use more storage space. We recommend this approach if storage is not a concern or you don’t already have software like python or $\text{\LaTeX}$ on your computer.
2. Install all requirements separately. More steps, but if you already have some of the required software installed, this will save disk space and might speed up editing.

Detailed instructions for both of these options is available in the [PreTeXt Guide](#).

With either approach, you will want a modern text editor to edit the PreTeXt source files. We strongly recommend using [Visual Studio Code](#), which has an extension for writing PreTeXt (`pretext-tools`) and will walk you through the docker container setup if you go that route.

Chapter 2

Workflow

Once you create content in PreTeXt, it is time to **build** it, **view** the result to make sure it looks good, and when you are ready, **deploy** your web output to a live webpage for others to marvel at.

The basic workflow in PreTeXt is to edit source files, build the project, preview the output, and repeat. Assuming you are working in a codespace, make sure you sync your files to GitHub regularly (see [Section 2.5](#) for details).

2.1 Prerequisites

In this chapter we assume you are working in a GitHub Codespaces, and have already initialized a PreTeXt project there. A few notes:

- If you do not have a **repository** for a PreTeXt project yet, go to <https://github.com/pretextbook/pretext-codespace>. On that page, below a short list of files, you will see a README that gives instruction for creating a project from that template.
- If you have a project already (on GitHub, in a repository), then to get back to the codespaces, navigate to the repository on GitHub.com. When you are on the page for your repository, you can restart or create a Codespace using the green (or blue) “Code” button.
- Most of the content below is also available as a walkthrough in VS Code. Open the Command Palette (or use the search bar at the top of the window and start with “>”) and search for “Welcome: Open Walkthrough...”. Then select the “Get Started with PreTeXt” option.
- If you prefer to work locally, almost all of the instructions in the following sections will be exactly the same if you use VS Code on your own computer (but you will need to ensure you have PreTeXt and other tools installed).

2.2 Build

Inside the VS Code window, have a `.ptx` file open (these are all inside **source** folder). You can build your entire project in a few different ways.

- Click the “PreTeXt” button in the VS Code status bar (just left of center on the bottom of the window), or use the keyboard shortcut `CTRL+ALT+p`, to bring up the PreTeXt Commands menu. Select “Build default target” from the menu. This will build your files and put the output inside the `output/course` folder.
- You can type `pretext build course` from the terminal (this uses the CLI directly). If you don’t see a terminal window at the bottom of the screen, you can open it with `CTRL+`` or go to “Terminal” menu and select “New Terminal”.

That’s it. If you get a message proclaiming “Build successful!”, you can jump to the next section.

Note 2.2.1 All shortcuts listed in this guide assume you are using Windows or Linux. If you are using a Mac, you will need to use `CMD` instead of `CTRL`.

If your document contains some more complicated elements, you might need to **generate** them for them to show up. The elements that require this are (depending on what your build target is):

- `<latex-image>`
- `<prefigure>`
- `<asymptote>`
- `<sageplot>`
- `<youtube>` (for thumbnail previews)
- `<webwork>`
- `<code-lense>`

All such elements will get generated automatically when you build. If you don’t change them, then the next time you run a build things should be faster.

Note 2.2.2 Note that generating assets requires additional software, like $\text{\LaTeX}$ or Sage. The default Codespace setup has $\text{\LaTeX}$ installed, but not Sage. If you want to use `<sageplot>` images, or run into problems generating tikz images, look at the README that is included in your project.

2.3 Preview

You can check the output of what you built using the **View** command. Again, you can access this in multiple ways. If you use the PreTeXt menu and select “View full document”, you will get a choice of targets to view. At this point, you still want to view the `course` target.

If you like using the command line, in a terminal enter `pretext view course`. This should open a new tab in your browser that will display your book.

Once you have run the view command once and have a browser window open, any time you make a change to your source files, you simply rebuild (using the build commands from [Section 2.2](#)) and then refresh the browser window to see your changes.

An alternative way to preview your work is to use a tool called **CodeChat**. From the PreTeXt menu, select “Preview file with CodeChat.” This requires that you are looking at a specific PreTeXt file (and as you navigate to other

files, the preview will follow you, usually). You should see a preview in a new tab in your editor, right next to your source.

Note 2.3.1 Sometimes the view commands don't work well in Codespaces (forwarding ports is tricky). If you try to view and get errors, or the new tab that opens up does not show anything, you can try the following:

- On your codespaces tab, reload the web page. You shouldn't lose any work, but it will restart the web app that is showing VS Code.
- If that doesn't work, open a terminal ("Terminal" menu, "New Terminal", or `Ctrl+``) and type `pretext view -s` and then `pretext view`. The first command will stop any running web servers, and the second will start a new one that hopefully works better.
- If that's still not working, you can restart the codespace. This is basically like turning the computer off and on again. You can do this by going to the Codespaces button on the bottom left of the window, selecting "Stop Current Codespace". You should then be able to restart it.

2.4 Deploy

So you have worked tirelessly to prepare course notes or a book, built and previewed, and now you are ready to share the results of your efforts with the world. It's time to **deploy** your project.

With the codespace setup this is simple. From the PreTeXt menu, select "Deploy". This will automatically take the most recent build of your web target and host it through [GitHub Pages](#). Watch the output pane for a link to your published site. (It can take a few minutes for the site to get set up or updated; there should be another link to view the progress of the GitHub "action" that reports the progress.)

You can find a link to the site later by going back to your repository on GitHub. Click the "Settings" tab, and scroll down to the "Pages" section. There you will find a link to your site.

2.5 Important: Saving your work

Using codespaces will keep all your files "in the cloud" on GitHub's server. As long as you don't delete your codespace (which GitHub might do after a few days automatically, depending on your settings), your files will be saved there. However, you will want to **push** these files to your **repository** on GitHub to make this save permanent. This has the benefit of allowing collaborators to access your files as well (your codespace is unique to your account).

If you deploy your project, your changes will be saved and pushed to GitHub automatically. However, if you are not deploying, you will need to do this manually.

There is a *lot* to learn about **git** (the software behind GitHub), but luckily using VS Code lets you do everything you need using menus (you don't need to use the command line, unless you want to). Everything can be controlled using the **Source Control** view: it should be third from the top on the very left of the window, an icon with splitting paths, and likely a badge showing how many files you have changed.

Here are the basic concepts you need to understand.

- The program **git** keeps track of all the changes you make to files inside of your **repository** (in this case, the folder containing your project).
- Once you have edited your files and are happy with all of them, you tell git to track the set of changes as a **commit**. This creates a handy *breakpoint* you could return to if you want to go back to an earlier version. There are two steps to creating a commit (which you can often do all at once in practice):

1. You **stage** the files you want to update in the commit.

You **commit** the stage files including a **commit message**.

Doing this in two steps can be helpful if you want to commit only some of the files that have changed.

- Once you have one or more commits, you need to sync these changes with GitHub. To “upload” your changes, you **push** the repository. To download changes that you are someone else made, you **pull** the repository. In VS Code, you might have the option to **sync**, which does both of these at once.

Now, how do we do these things in VS Code? Start by looking at the Source Control view. You will notice a list of files that were changed. You can click on any of these to see what the changes are (you will see a side-by-side view of the original and updated version).

If you are comfortable staging and committing in one step, you can simply write yourself a short message in the textbox above the big green “Commit” button, and click the button. If you want to stage first, click the + next to each file under “changes” to stage them.

The green button should now turn into a “Sync” button. When you click that, it will do a quick pull and then a push, to sync changes with GitHub.

The only small point about using git is that not all files will be tracked. This is on purpose, since temporary files really should not be “remembered” using this version control setting. Which files or types of files are ignored by git is controlled by the “.gitignore” file in your repository.

In particular, we do not track the output of builds. Git is used to track progress on your source, which you build into output at any time. If you want others to be able to see the output of your work without building it themselves, you need to deploy your work.

2.6 Collaborating

If you want to work with colleagues or students on a PreTeXt project, you will each want to work inside your own codespace and use GitHub’s collaboration tools to manage sharing updates. Using Git and GitHub takes practice and there is lots to learn.

Until there are better suggestions in this book, check out [GitHub 4 Mathematicians](#), a free PreTeXt book, for all the details.

Chapter 3

Organization

In this chapter we explore how the PreTeXt course is structured, considering both how we organize the files that create the course, and how the course “book” is organized into chapters and sections. Then we will make our first edits to the course files.

3.1 Course Structure

The template for a new PreTeXt course assumes you will include most of the course documents inside a single PreTeXt **book**. This book will be split into **chapters** for different categories of course materials, and each chapter will be divided into **sections** representing individual course documents.

For example, there is a chapter for in-class activities, divided into **worksheet** elements (which become a type of section since they live inside a chapter). The way the course book is set up, each section gets its own webpage. This allows you to embed that section inside your learning management system (LMS) if you wish.

Here is how the template is structured by default, as seen in the output.

1. Course documents
 - (a) Syllabus
2. Course Notes
 - (a) Week 1
 - (b) Week 2
 - (c) ...
3. In-Class Activities
 - (a) Activity 1
 - (b) Activity 2
 - (c) ...
4. Handouts
 - (a) Handout 1
 - (b) Handout 2
 - (c) ...

5. Homework

- (a) Homework 1
- (b) Homework 2
- (c) ...

This is accomplished using the following source structure in the `main.ptx` file.

```
<book>
  <title>Course Title</title>
  <chapter>
    <title>Course Documents</title>
    <section>
      <title>Syllabus</title>
    </section>
  </chapter>
  <chapter>
    <title>Course Notes</title>
    <section>
      <title>Week 1</title>
    </section>
    <section>
      <title>Week 2</title>
    </section>
    ...
  </chapter>
  <chapter>
    <title>In-Class Activities</title>
    <section>
      <title>Activity 1</title>
    </section>
    <section>
      <title>Activity 2</title>
    </section>
    ...
  </chapter>
  etc.
```

Note 3.1.1 Actually, if you look at the `main.ptx` file, you will see something a little different: the individual sections are not listed since they are separated into individual files and included using `<xi:include>` elements. See [Section 3.2](#) for more details on how the source files are organized.

A big advantage of this structure is that you can cross-reference between different course documents easily. However, it is possible to organize the course differently, and it is likely you will want to exclude at least some of the files from the main course book. For example, if you write your quizzes and exams in PreTeXt, you probably want to be able to build these but not deploy them (at least until after the quiz or exam). Another case is slides, since slideshows are top-level PreTeXt documents.

3.2 File Structure

This section describes how the source files for the course are organized.

All of the content that you author go in `.ptx` files inside the `source` folder. By default the *entry* file for the main course book is called `main.ptx` (you can identify entry files because their root element is `<pretext>`).

While we could have authored everything inside this main file, it can be easier to *modularize* our source and keep individual sections (corresponding to particular course documents) in their own files. This makes it easier to focus on one piece of content at a time, and also makes it easier to reuse content (like sharing a handout between different courses).

The course template further organizes the source folder by creating folders for different types of files. So you will see a `syllabus` folder, a `notes` folder, an `activities` folder, and a `handouts` folder. Each of these folders contains the relevant source files for that type of course document. These can be organized in other ways, pretty much however you like, but if you move a file, you will need to change the path where it is included in the main file.

In the main file, each file is included using the `<xi:include>` tag. For example, the syllabus is included like this:

```
<xi:include href="./syllabus/syllabus.ptx"/>
```

(The `./` at the start of the path means to start looking in the same folder as the main file.)

You can include files like this anywhere in the project, including inside files that are themselves included. Two important things to make this work though:

1. The file you include must be a valid XML file, that has a single root element. It is possible to include other files (like tikz code, for example), but if you do that, then the `<xi:include>` element should have a `@parse` attribute with value `text`. That is,

```
<xi:include href="./path/to/file" parse="text"/>
```

2. The file that does the including needs to declare the `<xi>` `<namespace>` at the top of the file. This is done by adding the following attribute to the root element of the file:

```
xmlns:xi="http://www.w3.org/2001/XInclude"
```

See the top of the `main.ptx` file for an example of this.

Other files and folders. Besides your source files, a few other important files and folders are worth mentioning here.

- **publication** folder: This folder contains the publication files that control “publisher decisions” for your course, like color theme, what elements are shown and how, and other presentation details. The default publication file is called `publication.ptx`. We will explore how to customize this file in [Chapter 6](#).
- **assets** folder: Once you have external assets (like images) that you want to include in your course, you can put them in this folder (which you might need to create). You can create subfolders inside this folder to help organize things if you like.
- **output** folder: When you build your course, PreTeXt puts the generated output files (like the web version and the PDF) in this folder. You should not modify anything in this folder (it will be overwritten the next time you build the course).
- **project.ptx** file: This file contains project-level settings, as well as settings for each “target”, including the main course. We will discuss this file more in [Chapter 7](#).

3.3 First Edits

Let's make our course our own by updating some of the basic course information.

Change the course title. Open the `main.ptx` file (inside the source folder). The top of the file will have the following lines.

```

1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <!-- This is the main course file for ... . It is used to
      generate all the public facing course materials so
      those can be hosted online. -->
4  <pretext xmlns:xi="http://www.w3.org/2001/XInclude">
5      <!-- Include common macros etc: -->
6      <xi:include href="./docinfo.ptx" />
7      <!-- By creating a PreTeXt "book" for the course, we
      can split components into "chapters" -->
8      <book>
9          <title>Course Name (change me)</title>

```

Let's take it line-by-line. The first line is the XML declaration, which should be included in all PreTeXt files (and will be automatically added if missing when you format your code). This just identifies the file as XML.

Lines 3, 5, and 7 are comments. In XML a comment starts with `<!--` and ends with `-->`. Comments are ignored by PreTeXt and are just there to help human readers understand the source. You can change or delete these comments as you like.

Line 4 is the opening `<pretext>` element. This identifies this file as a PreTeXt document (all the way at the bottom of the file is the closing `</pretext>` element, indicating the end of the document). The `@xmlns:xi` attribute declares the XML namespace for XInclude, which is needed to use the `<xi:include>` element later in the file.

That is what happens on line 6, where we include the `docinfo.ptx` file that contains common macros and other information for the course.

Finally on line 8 we open the `<book>` element, which contains all the content for the course. And directly after this on line 9 (the first child of the `<book>`) is the `<title>` element, which contains the title of the course. Let's change that now.

It is the contents of this `<title>` element that will appear as the main title of the course in the generated output. Change the text `Course Name (change me)` to the actual name of your course. We will call our course *PreTeXt by Example*, so our updated line 9 looks :

```
<title>Demo PreTeXt Course</title>
```

Let's also update the subtitle (on line 11) to look something like this:

```
<subtitle>PTX 101 Spring 2026</subtitle>
```

Now we can rebuild the course (see [Section 2.2](#)) and see our updated title and subtitle in the output (go back to the tab where the course is open and refresh the page to see the changes, or run `pretext view` if you have closed that tab).

Changing Author Information. The next non-comment line in the `main.ptx` file is the inclusion of the `frontmatter.ptx` file. You can open this file by finding it in the explorer panel on the left side of the screen, or by clicking on the link in the `@href` attribute of the `<xi:include>` element, while holding down the `Ctrl` key (or `Cmd` on a Mac).

In the `frontmatter.ptx` file, we see the root element is `<frontmatter>`; since we included this file using the `<xi:include>` mechanism, we can only have one root element. The entire contents of this element (including the start and end tags) will be inserted into the parent document (`main.ptx` in this case) at the location of the `<xi:include>`.

```
<frontmatter xmlns:xi="http://www.w3.org/2001/XInclude"
  xml:id="frontmatter">
  <bibinfo>
    <author>
      <personname>Instructor Lastname</personname>
      <department>Mathematical Sciences</department>
      <institution>University of Templates</institution>
    </author>
    <date>
      Last Updated: <today />
    </date>
  </bibinfo>

  <!-- This displays the title page information -->
  <titlepage>
    <titlepage-items/>
  </titlepage>
```

The two child elements of `<frontmatter>` are `<bibinfo>` and `<titlepage>`. The first of these contains information about authors and other document data. Go ahead and change the author information here. You can also read the [Bibinfo section of the PreTeXt Guide](#) for additional content you can add to this element.

The `<titlepage>` element contains a **generator** element, `<titlepage-items>` which will automatically insert the parts of `bibinfo` that should go on a title page. So don't edit anything there.

Go ahead and rebuild the course and see the changes. At this point you might also want to see what happens if you remove the `frontmatter` element entirely (should a "course" have `frontmatter`?). You can return to the `main.ptx` file and comment out the `<xi:include>` element that includes the `frontmatter`.

Tip 3.3.1 Keyboard Shortcut for Comments. In VS Code, you can toggle comments for one or more lines with the `CTRL`+`/` shortcut (or `⌘`+`/` for Mac).

Next Steps. Now would be a good time to save our work. Commit and sync your repository as described in [Section 2.5](#).

Next up we will start adding components of the course, starting with a syllabus.

Demo Course Code

Completing the steps above should put your folder in the same state as the `2-first-edits` branch of the [Demo Course repository](#).

Chapter 4

Course Components

In this chapter we explore the main components that you might include in a course document created with PreTeXt. These include lecture notes, slides, in-class activities, homework sets, and so on.

4.1 Syllabus

The course template has set up the syllabus as a *section* of our course, inside the *Course Documents* chapter. This can be seen by looking at the `main.ptx` file, on the lines following the inclusion of the frontmatter file.

Listing 4.1.1 Fragment of `main.ptx` after the `<subtitle>`

```
<xi:include href="./frontmatter.ptx" />

<chapter xml:id="course-documents">
  <title>Course Documents</title>

  <!-- Course syllabus -->
  <xi:include href="./syllabus/syllabus.ptx" />

  <!-- Other documents could go here, like a schedule,
       project description, etc. -->
```

We also see that the file for the syllabus is inside a `syllabus` folder, and is called `syllabus.ptx`. Opening this file we see that the root element (there can only be one) is indeed a `<section>`, and that we have titled the section *Syllabus*.

Listing 4.1.2 First part of the `syllabus.ptx` file template

```
<?xml version="1.0" encoding="utf-8"?>

<section xml:id="syllabus"
  xmlns:xi="http://www.w3.org/2001/XInclude">
  <title>Syllabus</title>

  <introduction>
    <p>
      Welcome to what promises to be an exciting and fun
      semester of ...
    </p>
  </introduction>
```

The next element provided by the template is an `<introduction>`. It contains a single paragraph, but you can add more. Each paragraph is enclosed in a `<p>` tag (think of this as the analogue of text separated by a blank line in $\text{\LaTeX}$).

Divisions and their Structure

Let's pause and briefly consider how **divisions** work in PreTeXt, and what their structure can be. See also the section on divisions in the [PreTeXt Guide](#).

The main divisions of a PreTeXt book are either **chapters** or **parts** that themselves contain chapters. Chapters can contain **sections**, which can contain **subsections** (there is even a **subsubsection** that can belong to a subsection). The same structure applies to a PreTeXt **article**, although there the divisions start with sections (no chapters or parts).

For each sort of division, there are exactly two choices about their structure.

1. The division can contain smaller divisions, exactly one level lower than itself (so chapters can have sections, sections can have subsections, but chapters cannot have subsections as direct children).
2. The division can contain only content, with no further divisions.

In particular, you cannot mix and match these approaches. It is impossible for a section to have subsections and *also* content outside of a subsection, for example.

However, there are situations where it makes sense to start a subdivided section with some sort of “introduction”, and perhaps end it with a conclusion. In this case, content that comes before the first subdivision *must* be placed inside an `<introduction>` (and content after the last subdivision must be inside a `<conclusion>`).

An introduction and conclusion should not contain “real” content though. In particular, don't include blocks that could get a number in these.

For the syllabus section, there are subsections, so the short introduction before the first subsection is inside an `<introduction>`. After the close of the introduction, we start the first `<subsection>`.

Course Info: Description Lists and text “styling”

The first subsection of the syllabus has a title of “Course Info” (which we could certainly change if we wanted). Perhaps we should start with some basic info, like who the instructor is, where their office is, what textbook will be used, and so on. One way we can organize this list of information is to use a **description list**.

PreTeXt offers three sorts of lists: unordered (using tag `<ul>`), ordered (`<ol>`), and description (`<dl>`). The first two are analogous to $\text{\LaTeX}$'s `\begin{itemize}` and `\begin{enumerate}` respectively. Each **item** of the list is wrapped in the `<li>` tag (for **list item**).

For description lists, since we are providing a description of a particular thing, each `<li>` gets both a `<title>` and a paragraph (using the `<p>` tag).

Listing 4.1.3 Description list included in a `<p>` tag after some text.

```

<dl>
  <li>
    <title>Instructor</title>
    <p>
      Prof. Lastname, Office Location,
    <url
      href="mailto:prof.lastname@example.edu">prof.lastname@example.edu</url>.
    </p>
  </li>

  <li>
    <title>Student Hours</title>
    <p>
      TBD
    </p>
    <p>
      <alert>Important</alert>: I <em>want</em> to see you
      in student hours,...
    </p>
  </li>

  <li>
    <title>Class meets</title>
    <p>
      course times and location.
    </p>
  </li>
</dl>

```

Of course you can edit the titles and `<p>` elements of each `<li>`, and add additional `<li>` as well. You will notice a few additional tags in the template that are worth mentioning.

- `<url>`** In the item for the instructor, the template suggests adding a link to the email (using the `href="mailto:..."` hyperlink). The `<url>` element is how you can link to *external* documents. You provide the external location as the value of the `@href` attribute. If you would like the text for the link to be something other than the url itself, you can put that as the *contents* of the `<url>` element (i.e., between the opening and closing `<url>` tags).
- `<alert>`** The second paragraph of the “student hours” item starts with an `<alert>` element. In some output formats, text inside an `<alert>` will be rendered as bold, but remember, we are not using this tag because we want the words “Important” to be bold, but because we think the reader should be alerted to this word.
- `<em>`** Similar to the `<alert>` element, an `<em>` element is intended to designate that the contents is **emphasized**. This is usually displayed as italic text.
- `<pubtitle>`** The title of the course textbook can be wrapped in `<pubtitle>` tags. I believe this will style the title as italic as well, but honestly I’m not sure. This is the great thing about PreTeXt: I don’t need to know the “correct” way to style the title of a textbook, I just need to say that the title *is* the title of a textbook.

Keep going

The next tag you will see in the template is a `<paragraphs>`. This is sort of like a division, although it can go at any “level” and doesn’t need to follow the same rules about structured divisions we mentioned earlier. It would have been fine to use `<subsubsection>` instead of `<paragraphs>`, but then we would have needed to decide what goes in the `<introduction>` of each `<subsection>`.

The remainder of the syllabus template is divided into additional subsections and paragraphs. This should be sufficient for any organization you need in a syllabus. My general approach is to think of each subsection as a major divider, with paragraphs inside each subsection to break up the content into manageable pieces. For subdivisions of these pieces, I use description lists, like we did for the parts of course info above.

One mechanism we have not yet mentioned though is using a table, which could be useful for describing how grades will be assigned (based on points or percentages). See the [Tables and Tabulars](#) section of the PreTeXt Guide to learn about these.

Finally, you might consider breaking the content of the syllabus across multiple files. This is especially useful if all your courses will have the same **course policies** subsection, for example. Having a separate file that you `<xi:include>` in all your syllabi can make copying that between courses easier.

Demo Course Code

Completing the steps above should put your folder in the same state as the `3-syllabus` branch of the [Demo Course repository](#). There you will see examples of a completed syllabus, including a table for grading breakdown and an external file for course policies.

4.2 Notes

Coming soon.

4.3 Handouts

Coming soon.

4.4 Assignments

Coming soon.

4.5 Slides

Coming soon.

Chapter 5

Content

You will likely want content beyond text in your course materials. In this chapter we explore how to include images, videos, interactive elements, and more in your PreTeXt course documents.

5.1 Images

Coming soon.

5.2 Video

Coming soon.

5.3 Interactive Exercises

Coming soon.

5.4 Tables

Coming soon.

Chapter 6

Publishing

By **publishing**, we mean here the process of finalizing the presentation and look of the output you get from PreTeXt. We stress again the philosophical distinction between **author** and **publisher** in PreTeXt, since there is great value in focusing on just one of these two types of tasks at a time.

Another aspect of the publishing process is that of creating the physical book and making it available in bookstores, or making the web version available on your own website. The latter can be done automatically through GitHub pages using the `pretext deploy` function, but perhaps you also want to have a separate landing page that links to a version of the book you are selling on Amazon.com. However, these steps are beyond the scope of this guide (as for suggestions on the PreTeXt-support group if you would like some assistance getting started with these steps).

For this chapter we will assume you are trying to produce a single **version** of your book and want to control the presentation and look of that version. You can still produce this version in multiple **formats** (web, pdf, epub), and which you want to produce might change what you do slightly.

6.1 The publication file

PreTeXt controls publisher options in a **publication file**. The default location of this file is in the folder `publication` and is itself called `publication.ptx`. Open this file and take a look.

What you will see is more XML, but a different set of tags than what you would write in your PreTeXt source. The version of the publication file that came with your codespace was (at the one time) the almost complete list of options with default values provided. This should make it easy to modify values and experiment with different settings.

As new settings are introduced, you can find documentation in the [Publication File Reference](#) part of the guide.

Remark 6.1.1 Eventually you may want to have different versions of your document, and you can do this with different publication files.

Next we will dive deeper into the different sorts of things that can be configured with this file.

6.2 Showing and Hiding things

We have seen that some **blocks**, like proofs, solutions, examples, are sometimes *hidden* behind a **knowl**: when you click on the title, the content expands to reveal itself. Most of the blocks can be “known” or not.

To edit these settings, look for the element `<html>`, and inside that `<knowl/>`. This element has a number of **attributes** (the PreTeXt guide marks attributes with the `@` symbol, by the way). You can specify whether elements should be known or not by changing the value of the attributes between yes and no.

Whether exercises are known depends on the type of exercises. PreTeXt distinguishes five types of exercise-like elements, since they have hints/answers/solutions that you might want to hide:

- inline, which show up as “checkpoints”; these are exercises mixed with other content in a divisions.
- divisional; these are exercises that belong to an `<exercises>` division.
- project; these are the entire project-like element like `<activity>`, `<investigation>` etc.
- worksheet; essentially a project but its a division itself, and can contain spacing for students to work in when printed.
- readingquestions; these are questions in a `<reading-questions>` division. These were exercises designed to get a text box that students can type short answers to questions at the end of a section.

You can control whether the entire exercise-like element is known, but their hints/answers/solutions are always known (no spoilers!). The only solution you can choose to not knowl is that of an example, and that is controlled by these knowl switches.

You can, however, control which parts of exercises show up at all. This is set in a different section of the publication file: `publication/common/exercise-inline` (that is, in the `<common>` block in the `<exercise-inline>` element). You can replace `exercise-inline` by any of the five exercise types. Then as attributes, specify yes or no for the `@statement`, `@hint`, `@answer`, `@solution`.

This controls what shows up in the output, in all formats, at the position that the exercise was authored. There are also ways to redisplay any parts of these exercises later, to give a list of all solutions in the back of the book, say. See [Exercises and Solutions](#) from the guide.

6.3 Numbering

There are a few settings that control how things are numbered. Most of these live inside the `<numbering>` element of the publication file.

- The `<division/>` element and its attributes says how “deep” the numbering goes (i.e., do you put numbers on subsection?). This is set as a natural number as the value of the `@level` attribute.
- Also in this element you can specify whether parts are **structural** or **decorative**. This essentially says whether chapter number should restart in each part.
- You can also set the number of the first chapter here using the `@chapter-start` attribute.

- There are four different types of elements that can be numbered independently: **blocks**, **projects**, **equations**, and **footnotes**. You can control how specific the numbering are for each of these by giving the corresponding element a `@level` attribute.

6.4 Formatting

While there are some ways to control the look of the pdf, that is beyond the scope of this guide. Web output can be controlled more easily, selecting from a small number of predefined styles.

This is a feature that is currently under active development, so we will not say much here, but instead refer you again to the [Publication File Reference](#) part of the guide.

Chapter 7

Managing Your Project

7.1 File and directory structure

Here we describe the file and directory structure of a project. PreTeXt allows a fair amount of flexibility in how you structure the project, but we believe the following are best practices.

source folder	Contains all your <code>.ptx</code> files that hold the content of your document. A new book starts with just a single file, but later we will see how to modularize the source to make organizing it easier.
publication folder	Contains your <code>publication.ptx</code> file or files, which define the publication-specific information about your document, as we saw in Chapter 6
assets folder	Put any images or other static files that you will include in your document here. This does not include images that you describe inside your source (like <code><latex-image></code>). You can have subfolders as you like, and if you refer to these files in your source, you do <i>not</i> use “assets” as part of the file name (PreTeXt knows where to look, since this is specified in the <code>publication.ptx</code> file.)
generated-assets folder	This is a folder that PreTeXt automatically creates and fills with assets that it generates from your source. You shouldn’t edit anything in this folder. It is not tracked by git by default.
output folder	Another folder created by PreTeXt. It will contain the results of <code>pretext build</code> . In general, you should not touch anything in this folder. Not tracked by git by default.
project.ptx file	This is the project manifest file, which you use to manage the different builds of your project. We will describe how to use this in more detail below.

There are a few other files that you might see in a project, such as `requirements.txt` and `.gitignore`. Don’t worry about these for now.

When your project grows, you will likely want to separate your `main.ptx` source file (inside the **source** folder) into multiple `.ptx` files. For example, you might want a single file per chapter, and even a separate file for each section. You can do this by using the `<xi:include>` tag. For example, if you have a

file `source/chapter1.ptx` that contains the first chapter of your book, you can include it in your `main.ptx` file like this:

```
<xi:include href="./chapter1.ptx" />
```

In the top level tag of the file in which you use `<xi:include>` (in this case, `main.ptx`), you must add the following attribute:

```
xmlns:xi="http://www.w3.org/2001/XInclude"
```

Then in the `chapter1.ptx` file, you would start with the standard `<?xml version="1.0" encoding="UTF-8"?>` followed by the top-level tag `<chapter>`. There can only be one top level tag in this file. A second chapter would need to be its own file and `<xi:include>` it separately.

More information can be found in the [Modular Source Files](#) section of the guide.

7.2 Project.ptx

You will likely want to build your source file into multiple output formats. How do you tell PreTeXt what outputs you want? You keep track of this, and some other information, in your `project.ptx` file.

The `project.ptx` file is a **project manifest** file. It is used to manage the different builds of your project. It is a `.ptx` file, but it is not part of the content of your document. It is used to tell PreTeXt how to build your document.

For now, take a look at the `project.ptx` file in this project. You will see that there are multiple **targets** each with a **name**, and a specified `<format>`, `<source>`, `<publication>` (file), and `<output-dir>` (directory). The `<source>` and `<publication>` are probably the same for all targets, but they don't have to be.

For a complete description of this file and its use, see <https://pretextbook.org/doc/guide/html/processing-CLI.html#cli-project-manifest>.

7.3 Versions

7.4 Building subsets

7.5 Codespaces, GitHub, and GitHub Pages

Chapter 8

Converting To PreTeXt

Suppose you have materials written in another format, such as MS Word or L^AT_EX, and would like to convert these to PreTeXt. Alas, there is not a perfect way to make this happen, but what follows is some general advice to get you started. We would love to hear what sort of workflow you would like to see so we can improve this experience.

Warning 8.0.1 Your goal in doing a conversion should be to move all your materials into PreTeXt and then always work from the PreTeXt source, converting back to other formats as needed. It is simply not feasible to keep converting to PreTeXt, since this process is not automatic.

8.1 Using Pandoc

If you have a number of individual documents that you would like to convert, from pretty much any format, consider trying [Pandoc](#). This command-line tool can read in lots of different formats and output in lots of different formats. Unfortunately, PreTeXt is not a default output format, so you need to use a custom **writer**.

The pretext-tools VS Code extension has a shortcut for using the custom writer and pandoc, assuming you have pandoc installed (which you will if you use a codespace). You will need to upload the file you want to convert, either by dragging it to the Explorer menu in VS Code. Then open the Command Pallet with CTRL+SHIFT+P and start typing “pretext: convert file to pretext” and select the item that shows up. You will be prompted for the name of the file you want to convert. The file will be converted and opened in a new tab. You can then copy/paste the contents into your main documents.

You can also use Pandoc outside of VS Code. Install pandoc from their website. Then download the this [writer](#) and put it somewhere you will remember. Documentation for how to use this tool is available on its [GitHub repository](#).

8.2 Using the community

If you have a mostly complete project, written entirely in not-too-customized L^AT_EX, David Farmer has volunteered to help you convert the entire project to likely 90% correct working PreTeXt. See <https://pretextbook.org/conversions.html> for details.

8.3 By hand

If you have some familiarity with Regex, you can use VS Code's find/replace tools to clean up text that you copied/pasted into your document. For example, if you entered `\$(.*?)\$` in the search bar (with the “.” setting selected), and put `<m>$1</m>` in the replace box, you can easily replace all math in your document with the correct syntax.

Appendix A

Getting Help

Here we collect a number of useful resources to help you when you are stuck. The official [PreTeXt site](#) has lots of resources, but we understand it can be overwhelming.

Official documentation. Note that the official [PreTeXt Guide](#) can be hard to use because there is so much stuff in it. Additionally, some of the documentation is out of date. Still, if you know where to look, it is a great resource.

Here are some sections that we find especially helpful:

- [Basics Reference](#): A listing of the main elements of PreTeXt including snippets of the code that create them. This is one of the few places in the guide that has examples of the markup.
- [Publication File Reference](#): When you are ready to start changing how your output looks, you can use the **publication file**, which is described in this part of the guide.
- [PreTeXt Schema](#): The official list of elements and where they can go is given in the PreTeXt Schema, which is described here. Also you can check out the [schema browser](#) to actually view the schema.
- [Getting PreTeXt](#): If you want to install PreTeXt on your own computer, this early part of the guide gives you directions. It should be updated with information on CodeSpaces soon as well, if you need a refresher.

Finally, note that the search in PreTeXt now works really well, and searching for a feature will usually get you pointed in the right spot.

Examples. The [Examples](#) page on the PreTeXt site contains a number of useful live examples. Links are provided to web, pdf, and source (on GitHub). For some of the examples, there is also an *annotated* version available. We find these especially helpful since you can “view source” to see exactly how each bit of the example was marked up in code.

Here are some of the most useful such examples:

- [Sample Book](#): This annotated sample book contains a section on [interactive exercises](#). The PreTeXt developers use this book for testing, so you can see the latest (sometimes experimental) features available.
- [Sample Article](#): Not particularly well organized (it is also a proving ground for developers) but this contains almost every variation of every feature of PreTeXt. Using the search and “view source” makes this an invaluable resource.

Community Support. There is a very active google group for support: [pretext-support](#). You should also subscribe to the low-traffic [pretext-announce](#) to get updates.

This spring we will host daily virtual “drop-in” sessions to support authoring and development of PreTeXt. Information will be posted to the pretext-announce google group.

Appendix B

Copyright and Licensing

© 2023 Steven Clontz and Oscar Levin.

This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit [CreativeCommons.org](https://creativecommons.org/licenses/by-sa/4.0/)

Appendix C

Acknowledgement

We would like to thank the [American Institute of Mathematics](#) for sponsoring us to present this Professional Enhancement Program at the 2023 Joint Mathematics Meeting.